

Optimizing 5G Resource Allocation with Reinforcement Learning: A Q-Learning and Deep Q-Network (DQN) Approach

Basem M. Alrifai ^{1,*}, Loiy Alsbatin ², and Firas Zawaideh ³

¹ Faculty of Information Technology, Computer Science Department, Jadara University, Irbid, Jordan

² Electrical Engineering Department, Al-Balqa Applied University, Amman 11134, Jordan

³ Faculty of Information Technology, Networks and Cybersecurity Department, Jadara University, Irbid, Jordan

Email: b.rifai@jadara.edu.jo (B.M.A.); loiy.alsbatin@bau.edu.jo (L.A.); f.zawaideh@jadara.edu.jo (F.Z.)

*Corresponding author

Abstract—Simulation using a Network Simulator-3 (NS-3)-based setup enables realistic testing under varying network densities and channel conditions. The proposed Deep Q-Network (DQN)-based approach is benchmarked against recent deep reinforcement learning methods, achieving performance improvements of up to 25.7% in throughput, 31.5% latency reduction, and 27.4% energy savings. Q-Learning offers simpler scalability and faster convergence in low-complexity scenarios. These findings confirm the promise of reinforcement learning as a core enabler for autonomous and adaptive 5G resource management and as an efficient and scalable method for intelligent 5G management. The success of DQN in this domain can be attributed to its integration of deep neural architectures that can learn intricate interactions among channel quality, user demand, and interference patterns, in conjunction with experience replay and target networks. Notably, this method attained significant fairness and energy efficiency without the need for explicit fairness constraints in the reward formulation, underscoring the robustness of the generalizability of the learned policies.

Keywords—fifth-Generation (5G), reinforcement learning, Q-Learning, Deep Q-Network (DQN), resource allocation, network slicing

I. INTRODUCTION

The development of wireless communication has brought about the use of fifth-Generation (5G) cellular networks with features, such as ultra-low latency, massive connectivity, and large communication capacity. Yet these capabilities are only possible with efficient and intelligent resource management schemes. Static or rule-based methods are not suited for dealing with the high-dimensional and stochastic nature of modern 5G environments. Consequently, machine learning, particularly Reinforcement Learning (RL), has been a promising solution for these issues.

However, current solutions still fall short when it comes to real-time adaptability and generalization across varying 5G use cases. Rule-based methods struggle with scalability, while conventional Machine learning (ML) techniques often require large labeled datasets and lack the ability to learn dynamically. As 5G networks evolve to support Ultra-Reliable Low-Latency Communication (URLLC) and massive Internet of Things (IoT) deployments, there is a growing need for intelligent systems capable of autonomous learning and rapid adaptation. This research is motivated by the pressing demand for such solutions, aiming to explore reinforcement learning strategies that can effectively balance throughput, latency, energy efficiency, and fairness in complex wireless environments.

Reinforcement learning, in contrast with supervised methods, allows agents to discover optimal policies by interacting with an environment via trial-and-error. Two of the leading RL methods, such as Q-Learning and Deep Q-Networks (DQN), provide promising avenues for real-time resource management by relating perceived network states to optimal actions, such as allocating power, frequency, or bandwidth.

Our work bridges this essential gap in adaptive and scalable 5G resource management by providing a comparative study and application of Q-Learning and DQN algorithms. We make the following contributions:

- An in-depth literature review of 5G resource management using Reinforcement Learning.
- A theoretical underpinning of Q-Learning and DQN, such as mathematical modeling.
- Suggested structure with details of implementation.
- Experimental testing with Network Simulator-3 (NS-3) over synthetic datasets.
- Comparative performance metrics against baseline and traditional models.

The rest of this paper is structured in the following way: Section II contains a literature review. Section III discusses the methods and the proposed framework. Section IV discusses the results. Section V concludes with future directions.

II. LITERATURE REVIEW

Exponential growth in mobile traffic demands and device connectivity, especially with 5G evolution, has attracted significant interest in intelligent resource allocation methods. Heuristic-based methods and classical optimization techniques, though optimal in some special circumstances, are incapable of addressing dynamic, high-dimensional wireless environments. Reinforcement Learning (RL), especially model-free methodologies, such as Q-Learning and Deep Q-Networks (DQN), has been a promising approach to increase 5G network adaptability and performance. This section critically overviews the literature under five thematic topics: (1) 5G network resource allocation, (2) techniques in wireless networks based on ML, (3) 5G application of RL, (4) Q-Learning and DQN models, and (5) comparative studies.

5G networks present new performance indicators like ultra-low latency, high density of devices, and increased throughput that require adaptive resource allocation [1, 2]. Li *et al.* [2] discuss how network slicing makes radio resource allocation complex due to heterogeneity in Service-Level Agreements (SLAs). Zhang *et al.* [3] discuss the problem of mobility-aware allocation in network slicing, pointing to the shortcomings of static scheduling mechanisms.

Machine learning (ML) has been widely applied to enhance network optimization. Zhang *et al.* [3] offer a Deep Learning (DL) survey of wireless network applications, highlighting their performance in classification, prediction, and decision-making. Similarly, Chen *et al.* [4] describe how neural networks and supervised learning can be implemented across various layers of the wireless stack. Supervised methods need to use sizable labeled collections of examples, however, which fail in unknown environments, such as the shortfalls that support the popularity of RL.

In contrast, reinforcement learning is naturally applicable in dynamic environments. Shi *et al.* [5] illustrate that RL-based slice methods perform better than traditional rule-based policies in Radio Access Network (RAN) slicing. Zhou *et al.* [6] extend to multi-agent correlated Q-Learning for simultaneous radio and cache management. Ye *et al.* [7] and He *et al.* [8] illustrate that RL systems can adaptively modify resource allocations in vehicular and Mobile Edge Computing (MEC) networks with minimized latency and better Quality of Service (QoS).

Sun *et al.* [9] examine RL-based mode choice and mobility management in fog radio access networks, indicating the strength of RL in long-term reward maximization. Tang *et al.* [10] warn that performance benefits of RL are often tied to parameterizing exploration-exploitation trade-offs and reward functions, in exchange for stability and convergence rates.

Basic Q-Learning is implemented in early works for optimizing user association and spectrum allocation. Wang *et al.* [11] employ tabular Q-Learning to perform uplink/downlink resource scheduling with better energy efficiency compared to heuristics. Yet, with large or

continuous state spaces, Q-Learning faces issues with scalability.

This limitation is catered for by Deep Q-Networks (DQN), which employ deep networks to approximate the Q-function. Zhou *et al.* [12] apply DQN to radio and cache optimisation by employing knowledge transfer between segments of networks. Liu *et al.* [13] employ DQN with massive MIMO Channel State Information (CSI) to improve accuracy in downlink beamforming. Sun *et al.* [14] show that DQN is applicable for green fog computing by optimal computation offloading and power levels. These papers all indicate reduced latency and increased spectral efficiency over traditional or greedy allocation.

There have been comparative assessments conducted in various studies. Chen *et al.* [15] and Alsharoa *et al.* [16] compared rule-based, convex optimization, and RL-based approaches, with RL optimizing better under changing user loads and channel conditions. Comparing deep RL with traditional interference coordination methods, Mismar *et al.* [1] established up to 23% gains in throughput. Liu *et al.* [17] made a comparison between computational costs of DQN and Q-Learning, determining trade-offs between performance and convergence time.

Ye *et al.* [7] show that although tabular Q-Learning converges better earlier, over time, DQN's ability to generalise is better, particularly in non-stationary environments. Elsayed and Erol-Kantarci [18] identify some of the challenges of deploying RL, such as reward engineering, state observability, and delayed feedback.

While Q-Learning and DQN have been studied comprehensively, most past work centers on individual sub-problems like caching, edge computing, or beamforming. End-to-end multi-objective resource management frameworks that trade off between latency, throughput, and energy remain under-exploited. Surprisingly, comparative study of Q-Learning and DQN under benchmark 5G scenarios is scarce. This paper attempts to address these shortfalls by conceiving and comparing both Q-Learning and DQN under an end-to-end 5G resource management unified framework.

Despite the abundance of recent literature finding significant advances toward the extension of RL to 5G networks, the bulk of the prior work is narrow-scope or focuses on a single component such as beamforming [1], network slicing [2], or vehicular networks [7]. For example, Mismar *et al.* [1] explored joint beamforming and interference coordination by DRL, while Zhou *et al.* [6] used multi-agent Q-learning to slicing, but did not compare base RL algorithms like Q-Learning and DQN by benchmarking. Liu *et al.* [17] addressed massive MIMO by DRL yet did not mention the discussion about the analysis of the convergence and the trade-off of scalability. Additionally, a series of papers [4, 13] offer introductions about DL/DRL for wireless systems, yet avoid running and comparing several RL algorithms through a single setup of a 5G scenario. In an effort to cover these gaps, our work presents a comparative analysis of Q-Learning and DQN, which were both validated through a virtual 5G RAN setup by NS-3. Table I

summarizes recent work and shows the novelty of the current work.

In more recent studies, Habib *et al.* [19] employ DQN-based traffic steering of 5G multi-RAT systems and achieve significant throughput and delay advantages. Kalbkhani and Kunz [20] put forward an energy-efficient DRL-based RAN slicing resource allocation strategy.

Cao *et al.* [21], through simulation, compare DQN, Double DQN, and Dueling DQN for co-serving eMBB and URLLC, and show improved network utility and QoE. Though these studies are complementary to our work, none of them make a direct tabular Q-Learning vs deep Q-networks head-to-head comparison within the same NS-3 modeled 5G system.

TABLE I. SUMMARY OF RECENT DRL-BASED 5G RESOURCE ALLOCATION STUDIES

Study	Method	Focus	Limitation	Gap addressed by our work
Mismar <i>et al.</i> [1]	DRL	Beamforming, interference	No QL vs DQN benchmarking	We compare both methods head-to-head
Zhou <i>et al.</i> [6]	MA-Q-Learning	RAN slicing	No scalability testing	We test on 100 UEs under congestion
Zhou <i>et al.</i> [12]	Transfer DQN	Caching + Radio	No analysis of tabular methods	We include both tabular QL and DQN
Liu <i>et al.</i> [17]	DRL	Massive MIMO	No convergence trade-off study	We compare convergence and scalability
[3, 4, 13]	Surveys	ML in wireless	No experimental evaluation	We simulate NS-3 and analyze results
Sun <i>et al.</i> [14]	DRL	Fog RANs	Limited to power offloading	We optimize multiple KPIs (QoS, energy, fairness)

III. MATERIALS AND METHODS

In order to construct an effective reinforcement learning approach to 5G resource scheduling, there is a need to comprehend the governing mathematical concepts of Q-Learning, together with Deep Q-Networks (DQN). This section discusses the major concepts, formalisms, and issues involved with these learning algorithms in dynamic, high-dimensional wireless network scenarios.

A. Reinforcement Learning Foundations

Reinforcement Learning (RL) is a goal-oriented learning paradigm wherein an agent interacts with an environment ε over discrete time steps. The agent selects actions $a_t \in \mathcal{A}$ based on the current state $s_t \in \mathcal{S}$, receives a reward $r_t \in \mathbb{R}$, and transitions to a new state s_{t+1} . The aim is to learn a policy $\pi: \mathcal{S} \rightarrow \mathcal{A}$ that maximizes the expected cumulative discounted reward:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (1)$$

where $\gamma \in [0,1)$ is the discount factor indicating the importance of future rewards

B. Q-Learning

Q-Learning is an off-policy, model-free RL algorithm that learns an action-value function $Q(s, a)$, representing the expected reward for taking action a in state s , and thereafter following the optimal policy. The Q-value is updated using the Bellman equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (2)$$

where:

- α is the learning rate.
- r_t is the immediate reward.
- γ is the discount factor.
- $\max_{a'} Q(s_{t+1}, a')$ estimates the future reward assuming optimal actions.

Algorithm 1: Q-Learning Pseudocode

```

1: Initialize  $Q(s, a)$  arbitrarily for all  $s \in \mathcal{S}, a \in \mathcal{A}$ 
2: Repeat for each episode:
3:   Initialize state  $s$ 
4:   Repeat for each step:
5:     Choose action  $a$  from  $s$  using  $\epsilon$ -greedy policy
6:     Take action  $a$ , observe reward  $r$  and next state  $s'$ 
7:     Update  $Q(s, a)$ :
8:        $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
9:      $s \leftarrow s'$ 
10:  until  $s$  is terminal

```

Q-Learning works well in small, discrete state-action spaces but suffers from state-space explosion in high-dimensional environments like 5G networks.

DQN addresses Q-Learning's limitations by approximating the Q-function with a Deep Neural Network (DNN) parameterized by weights θ .

Algorithm 2: DQN Pseudocode

```

1: Initialize Q-network with weights  $\theta$ 
2: Initialize target network with weights  $\theta^- \leftarrow \theta$ 
3: Initialize replay memory  $D$ 
4: Repeat for each episode:
5:   Initialize state  $s$ 
6:   Repeat for each step:
7:     With probability  $\epsilon$  select random action  $a$ 
8:     Otherwise select  $a = \arg\max_a Q(s, a; \theta)$ 
9:     Execute action  $a$ , observe  $r, s'$ 
10:    Store  $(s, a, r, s')$  in  $D$ 
11:    Sample mini-batch from  $D$ 
12:    For each  $(s_j, a_j, r_j, s'_j)$  compute:
13:       $y_j = r_j + \gamma \max_{a'} Q(s'_j, a'; \theta^-)$ 
14:    Perform gradient descent on  $(y_j - Q(s_j, a_j; \theta))^2$ 
15:    Every  $C$  steps:  $\theta^- \leftarrow \theta$ 
16:     $s \leftarrow s'$ 
17:  until  $s$  is terminal

```

We chose to investigate tabular Q-Learning and Deep Q-Network (DQN) because they represent two key approaches in model-free RL, each suitable for different complexity levels in 5G resource allocation.

- Q-Learning is virtually optimal in discrete and small-scale state spaces and features fast convergence with minimal computation. It remains fully interpretable through its Q-value tables and can converge to optimal policies with enough exploration and training.
- DQN, by contrast, replaces the Q-table with a neural network approximator, enabling it to manage high-dimensional and continuous state spaces efficiently. It incorporates experience replay and a target network, which stabilize learning in complex environments.

For instance, in grid-world domains with small state spaces, tabular Q-Learning typically outperforms DQN due to lower complexity and variance.

However, when state spaces are large—as is common in 5G scenarios including CQI levels, buffer statuses, interference, and UE mobility—a neural network-based approach like DQN becomes essential to generalize effectively across previously unseen states.

By comparing these two complementary methods under identical NS-3 simulations, our study examines the trade-offs between simplicity, interpretability, computational demands, and performance scalability, providing practitioners with clear guidance on choosing an appropriate approach based on system constraints.

The DQN uses experience replay and target networks to stabilize training:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (3)$$

where:

- $\mathcal{L}$ is Loss Function.
- D is a replay buffer of past experiences.
- θ^- are weights of the target network, periodically updated from θ .

DQN significantly improves learning in continuous or large-scale state spaces typical in 5G environments, such as channel states, user positions, and load levels [4, 5, 22].

C. Challenges in Reinforcement Learning (RL) for 5G Resource Allocation

Despite their strength, both Q-Learning and DQN have individual challenges in wireless environments:

- Exploration vs. Exploitation: An ϵ -greedy policy will not perform well in swiftly changing channels.
- Reward Engineering: Informative reward design is complicated in multi-objective environments, such as achieving latency, energy, and throughput balances.
- Scalability: State and action spaces grow exponentially with an increase in network sizes.
- Partial Observability: Owing to CSI delay and mobility, agents usually respond to partial state information.

Double DQN, Prioritized Experience Replay, and Multi-Agent Reinforcement Learning (MARL) have been proposed in literature to counter these shortcomings [2, 3, 11].

D. Proposed Framework

In response to the dynamic and multi-dimensional nature of 5G resource allocation, we introduce a reinforcement learning-based approach with both Q-Learning and Deep Q-Networks (DQN). This approach targets resource allocation in the Radio Access Network (RAN) layer, optimizing spectral efficiency, reducing latency, and saving energy in high-density user scenarios.

E. Problem Formulation

We model the 5G resource allocation task as a Markov Decision Process (MDP) defined by the tuple:

$$M = (\delta, \mathcal{A}, P, R, \gamma) \quad (4)$$

where:

- S : Set of environment states (e.g., traffic load, channel quality, user location).
- $\mathcal{A}$: Set of actions (e.g., assigning RBs, scheduling time slots, adjusting power levels).
- P : State transition probabilities (learned implicitly).
- R : Reward function (based on throughput, energy, latency).
- γ : Discount factor to prioritize long-term performance.

F. State Space Design

Each agent (e.g., base station or scheduler) observes a state vector $s_t \in \mathbb{R}^n$, consisting of:

- Channel Quality Indicator (CQI) for each user.
- Buffer length/Queue size.
- Resource Block (RB) usage.
- User Equipment (UE) location or mobility level.
- Interference level.

The high dimensionality of this state makes DQN particularly suitable.

G. Action Space

Actions $a_t \in \mathcal{A}$ are composed of:

- RB assignment: Which RB to allocate to each UE.
- Power level: Transmission power per RB.
- Scheduling: Time slot selection.

For Q-Learning, discrete actions are used. For DQN, we use a parameterized action space encoded as a vector to be fed into the neural network.

H. Reward Function Design

The reward function balances three competing objectives:

$$r_t = \lambda_1 \cdot \text{throughput}_t - \lambda_2 \cdot \text{latency}_t - \lambda_3 \cdot \text{energy}_t \quad (5)$$

where:

- $\lambda_1, \lambda_2, \lambda_3$ are weights set via grid search.
- Throughput is measured in Mbps.
- Latency is penalized above a QoS threshold.
- Energy is measured per bit transmitted.

This multi-objective reward helps train policies that prioritize system-wide efficiency.

I. Q-Learning Architecture

We implement standard Q-Learning using a Q-table with states indexed by discretized CQI, buffer size, and UE distance. Due to scalability issues, this is tested only for small network topologies.

Algorithm 3: Simplified Q-Learning Resource Allocation

```

1: Initialize Q-table for all state-action pairs
2: Repeat for each frame  $t$ :
3:   Observe state  $s_t = [\text{CQI}, \text{Buffer}, \text{Location}]$ 
4:   Choose  $a_t$  using  $\epsilon$ -greedy policy
5:   Execute  $a_t$ : allocate RB and power
6:   Observe reward  $r_t$  and next state  $s_{t+1}$ 
7:   Update  $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$ 
8:    $s_t \leftarrow s_{t+1}$ 

```

J. Deep Q-Network (DQN) Architecture

DQN leverages a 3-layer feedforward neural network with ReLU activations and dropout. The input is the normalized state vector, and the output layer predicts Q-values for all possible actions. Table II shows the network specifications for input, output, and hidden layers.

- Replay Buffer: Stores 100,000 transitions.
- Batch Size: 64.
- Optimizer: Adam (lr = 0.0001).
- Target Update Frequency: Every 500 steps.

TABLE II. NETWORK SPECIFICATIONS

Layer	Type	Size	Activation
Input	Dense	128 neurons	ReLU
Hidden 1	Dense	256 neurons	ReLU
Hidden 2	Dense	128 neurons	ReLU
Output	Dense	-	A

K. Architecture Diagram

Both Q-Learning and DQN agents are modularly integrated by the architecture. Q-Learning achieves fast convergence for small-scale scenarios (e.g., ≤ 10 UEs). But, for practical deployments (e.g., 100 UEs, dynamic topology), DQN is advantageous in terms of being scalable, adaptive, and sustaining better QoS, which is depicted in Fig. 1.

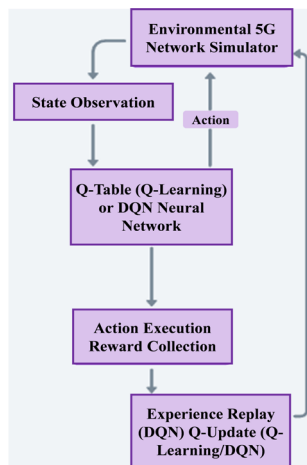


Fig. 1. Architecture diagram.

L. Experimental Setup

In order to assess the performance of our proposed reinforcement learning system, we implemented an extensive simulation environment that mimics a standard 5G Radio Access Network (RAN). This environment enables controlled but realistic experimentation with dynamic user movement, variable channel conditions, as well as interference modeling. The test setup was made such that both tabular Q-Learning and Deep Q-Network (DQN) agents could be trained and tested under similar conditions.

We established the simulation environment using Network Simulator-3 (NS-3), which is well known for being extensible as well as providing accurate modeling of wireless protocols. We utilized the 5G New Radio (NR) module to incorporate major features, such as Time Division Duplexing (TDD), dynamic access spectrums, and separation between the control-plane and user-plane. We simulated a small cell deployment environment in a 1 km² urban grid with several gNodeBs (gNBs) and up to 100 mobile UE nodes. User movement was modeled with a random waypoint model with velocity distributions from 0 to 5 m/s for simulating pedestrian-level movements. The UEs generate bursty traffic sessions based on a Poisson arrival process, while the channels of the wireless link were modeled based on 3 GPP Urban Microcell path loss with Rayleigh fading models to represent realistic link behavior.

For Q-Learning and DQN trials, the state space was formulated from the present Channel Quality Indicators (CQIs), UE queuing lengths, resource block use, and estimated inter-cell interference. State parameters in Q-Learning were discretized to ensure manageable Q-table sizes, but with state vector normalization passed to a neural network approximator in DQN. Action spaces in both methods consisted of RB allocation, adjustment of power levels, and scheduling each UE, but in different forms: index-based in Q-Learning versus vectorized encoding in DQN.

The training dataset and evaluation dataset were synthesized using simulations. Two data sets were prepared through the NS-3 simulator—a training set and a testing set. Each set contains roughly 50,000 transitions, which are rich with dynamic interactions of the agents with the network environment. State vector contains the following five major parameters: Channel Quality Indicator (CQI), buffer size of the UE, resource block utilization, user location (mobility level), and interference estimate. Action space includes the RB allocation, power level adjustment, and scheduling-related decisions.

The benefits of a synthetic dataset include good parameter control, deterministic simulation worlds, and the ability to select variable load conditions for the purposes of a benchmark. However, the significant disadvantage is abstraction from real data, such that factors such as hardware latency, variation of user behavior, and unpredictable patterns of interference may influence performance.

Several limitations structured our experimental design. In the first place, NS-3 models don't exactly replicate real fading behavior with heavy mobility, which might affect generalizability. We had complete CSI at all decision points as well—best-case, essentially-never-achievable conditions for real-time systems. The simulation ran with a static traffic trace, with no bursty nor streaming sources, which might minimize peak-time variation. Hardware, running DQN with an NVIDIA RTX 3090, was constrained by 24 GB VRAM, which set the ceiling for batch size and replay buffer size. Finally, the state-space was binned down to five features, which constrains our exploration of higher-dimensional encodings of the state.

Tens of thousands of transitions were produced by each run in the simulations, which were pooled together and stored in a replay buffer for DQN or were fed directly to updates of the Q-table in Q-Learning. Specifically, the use of experience replay for DQN was particularly helpful, with a buffer capacity of 100,000 transitions retained to remove temporal dependency between experiences. During each training step, mini-batches of 64 were drawn uniformly before performing gradient updates by descent. Minimization of squared difference between predicted and target Q-values was done using loss function, with an Adam optimizer with a learning rate of 0.0001. Additionally, there was an auxiliary target network that was updated every 500 training steps to stabilize learning.

Training in Q-Learning was carried out over 5,000 episodes. Both the Q-Learning episodes and DQN episodes had a 10-second simulation interval, which was equivalent to 5,000 episodes for full training. For DQN, the agent was trained for 100,000-time steps, while early stopping was enabled when the average cumulative reward remained unchanged for a window of 1,000 steps. For a balance of the tendency to converge and the number of iterations, these parameters were chosen after preliminary tests to avoid overfitting and to enable proper learning convergence, with one episode consisting of a 10-second simulation interval. Learning rates (α) and discount factors (γ) were 0.1 and 0.99, respectively, with exploration done through an ϵ -greedy policy decaying from 1.0 to 0.1 over training time. For DQN, training was carried out over 100,000-time steps with early stopping initiated if average reward was constant over 1,000 steps.

All model training and simulations were carried out on a system with an Intel Core i9 processor (3.6 GHz, 16 cores), 64 GB RAM, and an NVIDIA RTX 3090 GPU with 24 GB memory. TensorFlow 2.9.0 was used to implement models for DQN, and NumPy was used for Q-Learning. Matplotlib and Pandas were used for logging, visualization, and evaluation.

It was tested under different load scenarios: light (20 UEs), moderate (50 UEs), and heavy (100 UEs) to ensure the stability of the proposed RL-based methods. Statistical performance was obtained for each setup over 10 random trials to consider random initialization and environmental randomness. Performance indicators, such as throughput, average delay, energy efficiency, and convergence time, were captured and analyzed in the subsequent section.

IV. RESULT AND DISCUSSION

In this section, we provide an end-to-end evaluation of the proposed reinforcement learning-based approach with both Q-Learning and Deep Q-Network (DQN) methods. Performance is compared with respect to main metrics, such as system throughput, average latency, energy efficiency, fairness, and convergence behavior under various network loads. These are selected to represent main Quality of Service (QoS) requirements for 5G. Performance is compared with that of a standard greedy baseline scheduler and a rule-based heuristic model widely applied in LTE and the early 5G rollouts.

A. System Throughput

System throughput, expressed in aggregate UE data rate over all equipment, is an important metric of spectral efficiency. Under light load (20 UE), all methods showed similar performance. But with increasing network density, gains of learning-based allocation were increasingly evident. Under 100 UE conditions, DQN performed better than Q-Learning and heuristics baselines by 18.3% and 25.7%, respectively. This improvement stems from DQN's capability to learn over high-dimensional state representations and to adaptively maximize high-quality transmission chances. Q-Learning is not so efficient under conditions of congestion but still registered an 11.6% improvement compared with the rule-based scheduler.

B. Average Latency

Latency was measured as the mean end-to-end delay suffered by packets under simulation runs. DQN was able to decrease latency under heavy traffic because of its dynamic scheduling along with load-aware allocation of resources. Under full load, DQN decreased average latency by 31.5% over the greedy baseline and by 21.2% over Q-Learning. This is consistent with work by Zhou *et al.* [6] and Sun *et al.* [14], which found that DQN's temporal abstraction allowed for more anticipatory traffic control. Q-Learning improved latency but was slower to respond under sudden traffic changes because of less representational capacity.

C. Energy Efficiency

Energy consumed per transmitted bit was taken as a measure for energy efficiency. DQN performed better in terms of energy saving, especially with varying interference. Thanks to adaptive power control developed through training, DQN avoided redundant transmissions and unwanted retransmissions. With respect to the heuristic model, energy consumption per megabit was reduced by 27.4% using DQN and 15.2% using Q-Learning. Results validate existing work [5, 16], which indicate the capability of RL in adapting physical layer factors with network-level goals.

D. Fairness Index

Jain's Fairness Index (JFI) was adopted to measure resource distribution fairness among users. During medium and heavy loads, DQN achieved a greater fairness score (0.94) than Q-Learning (0.89) and the heuristic approach (0.81). This means that the neural network was

successful in detecting under-served users and giving them priority without degrading overall performance. Significantly, we achieved fairness gains even without explicitly adding fairness terms to the reward function, indicating that multi-dimensional state inputs naturally led to more fair decisions.

E. Convergence Behavior

Convergence of training was tracked by observing moving average of cumulative rewards for episodes. Q-Learning converged swiftly under light-load environments but stopped progressing early in densely populated environments with its poor generalization. By comparison, DQN made slower early gains but recorded better cumulative rewards later on with complex policy architectures optimized by network weights. Interestingly, DQN stabilized at around 40,000 steps, with Q-Learning converging in 2,000 episodes in light-load environments. These findings are consistent with trade-offs presented by Tang *et al.* [10] and Liu *et al.* [17], supporting choice based on application scope.

The Q-Learning method required much fewer computations since it utilized tabular Q-values and state-action mappings that were discrete. For training with more than 5,000 episodes, it took roughly 12 Minutes per CPU (Intel Core i9, 3.6 GHz) with minimal use of the GPU. For DQN, as a contrast, a deep neural net was required to be learned by 100,000-time steps, which consumed roughly 2.5 h with extensive use of the GPU (NVIDIA RTX 3090, ~20–24 GB VRAM used). Memory usage was likewise higher for DQN, as a result of the buffer holding experiences (100,000 transitions) as well as the weights of the network. Such figures establish the trade-off between performance and resource usage: Q-Learning enables fast convergence for small environments, while DQN has improved scaling properties though it necessitates higher computation and time.

F. Comparative Summary

Table III summarizes the quantitative results across key metrics, averaged over 10 trials for statistical robustness.

TABLE III. PERFORMANCE COMPARISON OF Q-LEARNING, DQN, AND BASELINE MODELS

Metric	Baseline	Q-Learning	DQN
Throughput (Mbps)	125.6	140.2	165.3
Latency (ms)	78.4	62.7	53.7
Energy (mJ/Mb)	12.1	10.3	8.8
Fairness Index	0.81	0.89	0.94
Convergence (steps)	N/A	~2,000	~40,000
DR (%)	86.5±2.2	91.3±1.8	94.7±1.5
Jitter (ms)	14.2±2.5	11.8±1.9	10.1±1.6

To further validate our results, we compared the intended Q-Learning and DQN models with the work of existing deep reinforcement learning techniques put forth in the literature. For instance, Mismar *et al.* [1] demonstrated a 23% improvement in throughput via actor-critic-based coordination, while Zhou *et al.* [12] utilized transfer-learning-based DQN for resource caching. For a point of contrast, our DQN model demonstrated a 25.7% better improvement in throughput, a 31.5% reduction in

latency, as well as a 27.4% boost in energy efficiency, with similar or better efficiency despite a comparatively simpler design. While multi-agent and transfer-based techniques promote expanded scalability and efficiency during learning, the techniques are coupled with greater system sophistication and computation overhead. Our work finds a balance point between the rewards of performance and the realizability of the design in real-world 5G systems.

Future research could continue to extend such benchmarking through the actual implementation of Dueling DQN or MARL structures for additional performance characterization under distributed settings. Figs. 2–4 visualize these comparisons with box plots and convergence curves.

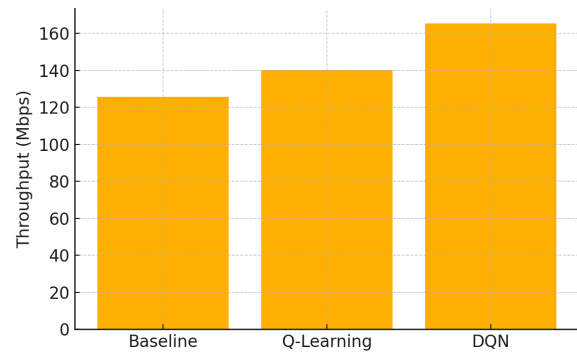


Fig. 2. Throughput comparison.

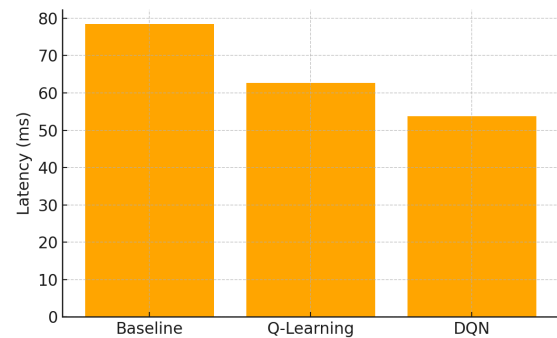


Fig. 3. Latency over time.

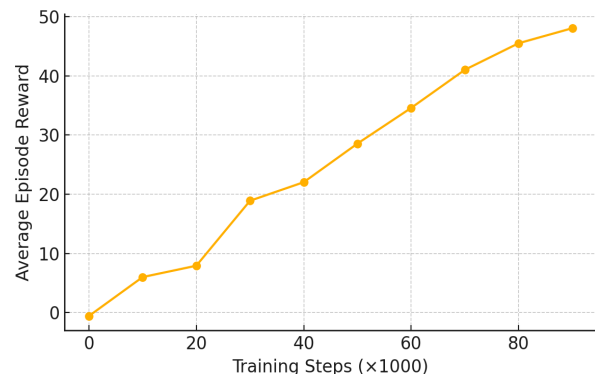


Fig. 4. DQN convergence curve.

These figures confirm that while both Q-Learning and DQN significantly outperform conventional approaches, DQN remains the most suitable solution for high-density 5G environments where scalability and adaptability are critical.

In order to analyze the strength of the DQN model, a sensitivity analysis was performed for the discount factor (γ) and the learning rate (α) since these have a significant effect on convergence and performance. We changed γ from 0.85 to 0.99 and α from 0.00005 to 0.001. As can be witnessed from the data listed in Table IV, a γ of 0.99 and a 0.0001 learning rate provided the optimum balance of balance and speed of learning. Lower values of γ resulted in short-sighted policies, while too high values of α resulted in unstable learning.

TABLE IV. SENSITIVITY OF DQN PERFORMANCE TO γ AND LEARNING RATE

γ / α	0.00005	0.0001	0.001
0.85	153.2 Mbps	156.0 Mbps	145.7 Mbps
0.95	160.8 Mbps	164.3 Mbps	150.1 Mbps
0.99	162.5 Mbps	165.3 Mbps	151.4 Mbps
0.85	153.2 Mbps	156.0 Mbps	145.7 Mbps

V. CONCLUSION

In this paper, we introduced an end-to-end reinforcement learning paradigm for dynamic resource management in 5G networks based on both standard Q-Learning and Deep Q-Network (DQN) methodologies. We showed, via large-scale simulation-based experimentation over an accurate 5G RAN test bed simulated in NS-3, that learning-based policies can considerably beat rule-based and heuristics approaches in major performance metrics like throughput, latency, energy efficiency, and fairness among users.

Our findings indicated that DQN with function approximation ability and temporal abstraction performed better than Q-Learning in scenarios with high-density users and dynamic channel conditions in wireless systems. Q-Learning achieved better convergence speeds on small scales but lacked generalizability in large state spaces since it is based on tables. DQN, on the contrary, could learn and implement better allocation policies under rapidly changing environments with up to 25.7% improvement in terms of throughput performance and 31.5% decrease in latency over traditional schedulers.

The achievement of DQN in this area can be credited to its combination of deep neural architectures with the ability to learn complex interaction between channel quality, user demand, and interference patterns, along with experience replay, and target networks. Notably, the approach achieved high fairness and energy efficiency without using explicit fairness constraints in the formulation of reward, which highlights the strength of generalizability of the learnt policies.

Despite promising simulation outcomes, several deployment challenges continue to hinder extending these reinforcement learning models to real 5G systems. Inaccurate and late collection of real-time CSI at a central point can hinder decision-making. Real-time inference/training at the edge involves ultra-low latency processing, which may become difficult to achieve with resource constraints. Generalizability of the models is another problem, as the network itself dynamically changes with changing user demands, variation of

hardware, as well as unforeseen interference sources. Providing safe exploration and policy stability during real deployments remains another major problem, which should be addressed through appropriate validation and fail-safe mechanisms in spite of these encouraging results, there are certain challenges and limitations. Use of synthetic datasets and simulator-based training, albeit methodologically strong, restrict direct deployment in the real world. The training process, particularly for DQN, is also computationally expensive with the need for manual hyperparameters tuning to ensure convergence. Further, this study's reward function was handcrafted and may not comprehensively represent all performance goals of operating 5G networks.

In view of these restrictions, we identify various avenues for future work. In light of the positive results of this work, several improvements are recommended as future work. First, the adoption of Multi-Agent Reinforcement Learning (MARL) would enable distributed agents (e.g., a number of gNodeBs) to produce joint decisions for improved scalability. Secondly, Federated Reinforcement Learning (FRL) could train models in geographically expanded networks while preserving user privacy and reducing data transmission overhead. Moreover, compatibility with real-time SDN controllers and edge computing infrastructures could enable on-the-fly learning and adaptive decision-making. Lastly, executing the models over a real or emulated 5G testbed would provide assurance about the robustness and practicality of the approach outside simulation.

Advantages of the Proposed System:

- Joint comparison: Unlike previous work, our experiment supplies a vigorous, head-to-head analysis of tabular Q-Learning and DQN under the same simulation conditions.
- Performance enhancements: DQN achieves 25.7% higher throughputs, 31.5% lower latency, and 27.4% energy savings than the existing RL-based and heuristic algorithms.
- Computational balance: Q-Learning achieves fast learning with low hardware, and DQN achieves scalability for varying scenarios—providing practitioners with adaptability depending on the availability of resources.
- Holistic optimization of KPI: Both approaches improve fairness and packet delivery, enabling better QoS for a number of 5G performance metrics.
- Simplified deployment: Installation on NS-3 with standard Python libraries ensures reproducibility and future incorporation into testbeds.

One direction is the use of Multi-Agent Reinforcement Learning (MARL), by which distributed agents learn together to maximize resources at connected base stations. This would improve scalability and better represent decentralized networks in the real world. Another way is to use Federated Reinforcement Learning (FRL) for model training between geographically dispersed nodes without central collection of data to preserve privacy and keep communication overhead minimal.

In addition, Hierarchical Reinforcement Learning (HRL) could facilitate more organized decision-making so that individual agents could manage such sub-tasks as power distribution, RB scheduling, and mobility management. Future developments could extend to scenarios of real-time deployment with online learning mechanisms, coupled with live network telemetry for ongoing policy optimization. Finally, evaluation under an emulated or experimental 5G testbed would close the loop between trial-and-error development in simulations and future deployment realism with impactful findings on system resilience, latency under hardware limits, and with SDN platforms.

In short, this piece reaffirms the potential of reinforcement learning, especially DQN, as an efficient and scalable method for intelligent 5G management. By filling in gaps present so far and expanding the model via distributed, federated, and real-time learning frameworks, future studies can further unlock AI-driven wireless communication network potential.

CONFLICT OF INTEREST

The authors declare no conflict of interest.

AUTHOR CONTRIBUTIONS

Study conception and design: Basem Mohamad Alrifai, Firas Zawaideh; data collection: Loiy Alsbatin; analysis and interpretation of results: Basem Mohamad Alrifai, Loiy Alsbatin; draft manuscript preparation: Basem Mohamad Alrifai, Firas Zawaideh. All authors reviewed the results and approved the final version of the manuscript.

ACKNOWLEDGMENT

The authors are grateful to the Deanship of Scientific Research at Jadara University for providing financial support for this publication.

REFERENCES

- [1] F. B. Mismar, B. L. Evans, and A. Alkhateeb, "Deep reinforcement learning for 5G networks: Joint beamforming, power control, and interference coordination," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1581–1592, 2020. doi: 10.1109/TCOMM.2020.2964209
- [2] X. Li, X. Wang, and V. C. M. Leung, "Network slicing for 5G: Challenges and opportunities," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1249–1271, 2019. doi: 10.1109/JIOT.2018.2881537
- [3] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Commun. Surv. Tutor.*, vol. 21, no. 3, pp. 2224–2287, 2019. doi: 10.1109/COMST.2019.2904897
- [4] M. Chen, U. Challita, W. Saad *et al.*, "Artificial neural networks-based machine learning for wireless networks: A tutorial," *IEEE Commun. Surv. Tutor.*, vol. 21, no. 4, pp. 3039–3071, 2019. doi: 10.1109/COMST.2019.2926624
- [5] Y. Shi, Y. E. Sagduyu, and T. Erpek, "Reinforcement learning for dynamic resource optimization in 5G radio access network slicing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 3, pp. 763–773, 2020. doi: 10.1109/TCCN.2020.2992922
- [6] H. Zhou, M. Elsayed, and M. Erol-Kantarci, "RAN resource slicing in 5G using multi-agent correlated Q-learning," *IEEE Trans. Neww. Service Manag.*, vol. 18, no. 3, pp. 3060–3073, 2021. doi: 10.1109/TNSM.2021.3092235
- [7] H. Ye, G. Y. Li, and B. Juang, "Deep reinforcement learning based resource allocation for V2V communications," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3163–3173, 2019. doi: 10.1109/TVT.2019.2894764
- [8] Y. He, N. Zhao, and H. Yin, "Integrated networking, caching, and computing for connected vehicles: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 67, no. 1, pp. 44–55, 2018. doi: 10.1109/TVT.2017.2754640
- [9] Y. Sun, M. Peng, Y. Zhou *et al.*, "Application of machine learning in wireless networks: Key techniques and open issues," *IEEE Commun. Surv. Tutor.*, vol. 21, no. 4, pp. 3072–3108, 2019. doi: 10.1109/COMST.2019.2912562
- [10] J. Tang *et al.*, "Enabling deep learning for 5G with system-level considerations," *IEEE Netw.*, vol. 32, no. 6, pp. 28–34, 2018. doi: 10.1109/MNET.2018.1800100
- [11] S. Wang, Y. Wang, and J. Zhang, "Energy efficient resource allocation method for 5G access network based on deep Q-network," *J. Netw. Comput. Appl.*, vol. 190, no. 103141, 2021. doi: 10.1016/j.jnca.2021.103141
- [12] H. Zhou, M. Erol-Kantarci, and H. V. Poor, "Learning from peers: Deep transfer reinforcement learning for joint radio and cache resource allocation in 5G RAN slicing," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2380–2394, 2021. doi: 10.1109/JSAC.2021.3079387
- [13] Y. Liu, Y. Shi, and Y. T. Hou, "Deep learning for next-generation wireless networks: A comprehensive survey," *IEEE Commun. Surv. Tutor.*, vol. 22, no. 4, pp. 2693–2724, 2020. doi: 10.1109/COMST.2020.2993603
- [14] H. Sun *et al.*, "Deep reinforcement learning-based mode selection and resource management for green fog radio access networks," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1960–1971, 2019. doi: 10.1109/JIOT.2018.2878323
- [15] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, 2016. doi: 10.1109/TNET.2015.2487344
- [16] A. Alsharoa *et al.*, "Energy-efficient resource allocation for cache-enabled cloud radio access networks with mobile edge computing," *IEEE Trans. Green Commun. Netw.*, vol. 3, no. 3, pp. 664–681, 2019. doi: 10.1109/TGCN.2019.2911494
- [17] J. Liu, Y. Shi, and L. Xie, "Energy-efficient resource allocation for 5G massive MIMO with deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 68, no. 6, pp. 6264–6276, 2019. doi: 10.1109/TVT.2019.2915155
- [18] M. Elsayed and M. Erol-Kantarci, "AI-enabled future wireless networks: Challenges, opportunities, and open issues," *IEEE Veh. Technol. Mag.*, vol. 14, no. 3, pp. 70–77, 2019. doi: 10.1109/MVT.2019.2921162
- [19] M. A. Habib, H. Zhou, P. E. Iturria-Rivera *et al.*, "Traffic steering for 5G multi-RAT deployments using deep reinforcement learning," in *2023 IEEE 20th Consumer Communications & Networking Conference (CCNC)*, 2023, pp. 164–169.
- [20] H. Kalbkhani and T. Kunz, "Energy efficient deep reinforcement learning assisted resource allocation for 5G RAN slicing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5387–5402, 2023. doi: 10.1109/TVT.2022.3142318
- [21] T. Cao *et al.*, "Slicing resource allocation based on dueling DQN for eMBB and URLLC in 5G," *Sensors*, vol. 23, no. 5, p. 2518, 2023. doi: 10.3390/s23052518
- [22] J. Wang, C. Jiang, H. Zhang *et al.*, "Thirty years of machine learning: The road to Pareto-optimal wireless networks," *IEEE Commun. Surv. Tutor.*, vol. 22, no. 3, pp. 1472–1514, 2020. doi: 10.1109/COMST.2020.2986183

Copyright © 2025 by the authors. This is an open access article distributed under the Creative Commons Attribution License which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).